

Building Production-Ready Agents

Securing the Agent: Identity

Amazon Bedrock AgentCore

Agenda

1. The Attribution Gap
2. Three-Identity Separation Model
3. Agent Identity Directory
4. Addressing OWASP Agentic Risks
5. Key Takeaways

1. The Attribution Gap

Why Agents Need Identity

- Agents act autonomously on behalf of users, breaking traditional IAM (Identity and Access Management) models.
- Without distinct identities, it is impossible to audit "who did what".
- **The Attribution Gap:** You must separate the human caller from the agent's workload role.

Agent Identity Model: Discussion

Your team is building a support agent that reads customer tickets, checks order status, and (with approval) issues refunds. You need to decide how the agent authenticates to downstream services.

DISCUSS

What identity should the agent present when it calls the order service and the refund service?

Agent Identity Model: The Options

Which of these would you choose?

1. **Reuse the caller's IAM role**: the agent impersonates the human user directly
2. **Single shared agent role**: one service account for all agent actions, across all users
3. **Three-identity separation**: distinct caller identity, workload role, and downstream credentials

Let's discuss the trade-offs of each approach before we look at what production teams do.

Agent Identity Model: The Answer

Production teams use three-identity separation.

Approach	Problem
Reuse caller's role	Agent can do everything the user can: no scoping. One compromised agent = full user access.
Single shared role	Every action looks identical in audit logs. Impossible to answer "who did what."
Three-identity separation	Caller identity answers who, workload role scopes what infrastructure, downstream credential scopes what data.

If your agent handles money, PII (Personally Identifiable Information), or clinical data, three-identity separation is not optional.

The Attribution Gap in Practice (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Agentforce in Slack):

- When an employee invokes an Agentforce agent from Slack, the design must distinguish the Slack user, the agent workload, and each downstream CRM or external-service credential.
- AgentCore Identity includes a Salesforce OAuth provider, but the application must still preserve caller context and enforce the authorized scope at every downstream action.

The Attribution Gap in Practice

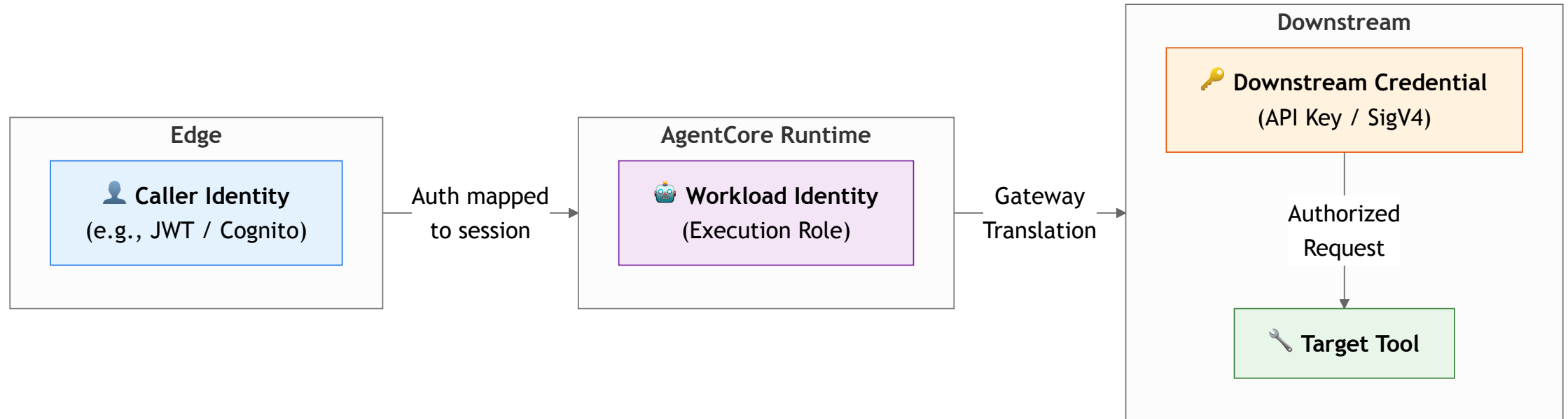
SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (order support):

- A shopper asks for order status, the Runtime acts with a workload role, and Gateway obtains only the credential needed for the read-only order API.
- A refund request is different: the verified shopper, approval decision, idempotency key, and refund-service credential must remain attributable rather than collapse into one agent identity.

2. Three-Identity Separation

Three-Identity Flow



1. Caller Identity

- Authenticated at the public edge (API Gateway, WAF).
- Typically via Amazon Cognito, Okta, or Entra ID.
- Defines who initiated the request.

Note: **WAF** stands for Web Application Firewall.

2. Workload Identity

- The IAM execution role used by the AgentCore Runtime microVM.
- Defines what infrastructure actions the agent is permitted to perform (e.g., invoking Gateway, emitting logs).
- **Never** use broad development policies in production.

3. Downstream Credential

- The credential presented by the agent to target tools via the Gateway.
- Often a scoped OAuth token or IAM SigV4 (AWS Signature Version 4) signature.
- Defines what business data the agent can access.

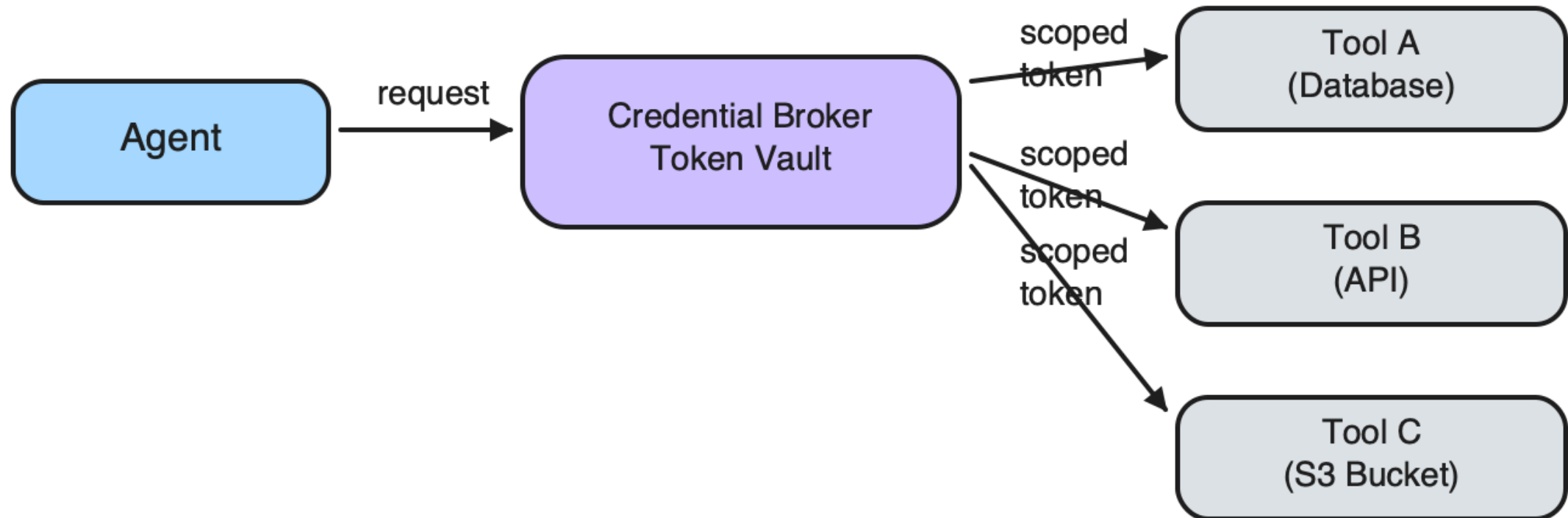
3. Agent Identity Directory

Identity Governance

- **Centralized Registry:** A single source of truth for all agent and workload identities.
- **Environment Agnostic:** Issues short-lived, task-scoped tokens.
- **Credential Broker:** Manages JWT (JSON Web Token), OAuth, and API keys securely in an encrypted token vault.

Agent Identity Directory

Agent Identity Directory



Broker intercepts requests and issues short-lived, task-scoped tokens

JWT, OAuth, and API keys managed in an encrypted vault

4. Addressing Agentic Risks

OWASP ASI03: Identity and Privilege Abuse

OWASP ASI03: Open Worldwide Application Security Project, Agentic Security Initiative, risk category 03.

- Agents are **identity aggregation points**.
- A hijacked agent inherits every credential it holds.
- **Control:** Use least-privilege IAM policies, short-lived tokens, and separate caller/workload/tool identities.

Defending Against Identity Aggregation (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Guided Determinism):

- Agentforce's Agent Graph enforces hard validation gates between workflow nodes. A refund agent cannot proceed from identity verification to refund authorization without passing a deterministic check. The model cannot talk its way past the gate.
- Apply the same boundary to credentials: each subagent or action receives only its own scoped authority rather than inheriting every permission in the workflow.

Defending Against Identity Aggregation (Retail and E-commerce)

SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (refund approval):

- The support agent may read an order and prepare a refund request but cannot issue the refund with its read-only workload role.
- A deterministic policy checks shopper ownership and amount limits; a named approver authorizes exceptions, preserving evidence for every financial action.

Deterministic External Controls

- Security must be enforced via infrastructure controls, not model prompts.
- Do not rely on "do not delete the database" in system prompts.
- Use IAM permissions, permission boundaries, and approval gates.

Prompts vs. Infrastructure

A refund agent's system prompt says: "Never transfer more than \$1,000." During a pentest, a prompt injection causes the agent to attempt a \$50,000 transfer.

- A) A stronger system prompt with all-caps emphasis would prevent this
- B) A downstream payment service enforcing the \$1,000 limit via IAM would prevent this
- C) Adding the limit to the agent's long-term memory makes it enforceable
- D) Logging the attempt and alerting security after the fact is the best defense

Chat Check

Type your answer (A, B, C, or D) in the chat window now.

Prompts vs. Infrastructure: The Answer

B: A downstream payment service with a \$1,000 per-transaction limit enforced by IAM would prevent this.

Prompts are suggestions. They can be ignored, forgotten, or bypassed through injection. IAM policies and service-level limits are deterministically enforced when correctly configured. The system prompt should describe desired behavior; the infrastructure should guarantee it.

5. Summary

Key Takeaways

- A production agent request traverses three identities: Caller, Workload, and Downstream Credential.
- The Agent Identity Directory manages these distinct, non-human identities.
- Correctly scoped IAM (Identity and Access Management) controls reduce blast radius; they do not replace application authorization or policy review.

Demo: Three-Identity Separation

REFERENCE ONLY

Run `python3 day1/demos/demo-identity-separation.py` (offline walkthrough).

- Traces caller → workload → downstream credential flow
- Shows OWASP ASI03 identity aggregation risk
- Compares Level 1 (shared keys) vs Level 2 (dedicated identities)

Predict the Blast Radius

An agent's workload role is hijacked via prompt injection (OWASP ASI03). Compare Level 1 (`bedrock:*` , `Resource: *`) against Level 2 (scoped to one model ARN (Amazon Resource Name) and one Gateway ARN).

What can the attacker do under Level 1 that Level 2 blocks?

- A) Nothing: both policies grant identical Bedrock permissions
- B) Invoke any foundation model or Bedrock resource in the account, not just its own
- C) Only affect CloudWatch logging: invocation is scoped identically either way
- D) Bypass Runtime microVM isolation, regardless of which IAM policy is attached

Chat Check

Type your answer (A, B, C, or D) in the chat window now, with one reason.

Demo: Least Privilege IAM

REFERENCE ONLY

Run `python3 day1/demos/demo-least-privilege-iam.py` (offline policy comparison).

- Compares broad development policy (bedrock:*) vs restricted production policy
- Shows IAM permission boundaries as structural controls
- Demonstrates deterministic security vs prompt-level instructions

Predict the Blast Radius: The Answer

B: under Level 1, the hijacked agent can invoke any foundation model and reach any Bedrock resource in the account; Level 2 confines it to one model ARN, one Gateway ARN, and AgentCore-only log groups.

`bedrock:*` on `Resource: *` grants no boundary at all. A compromised agent inherits the full blast radius of the account's Bedrock surface. The restricted policy scopes each permission to a specific ARN, so a hijack can only do the three things the policy names. **D** is the tempting wrong answer: IAM policy and microVM isolation are separate boundaries. A broad IAM policy doesn't break out of the microVM, but it does turn a contained compromise into an account-wide one. Least privilege doesn't stop the hijack; it limits what the hijack can reach.

Demo: CloudTrail Audit Evidence

REFERENCE ONLY

Run `python3 day1/demos/demo-cloudtrail-audit.py` (synthetic audit fixture).

- Shows how three-identity separation creates attributable audit logs
- Maps action → workload identity → caller identity
- Covers audit best practices for agentic systems

End of Identity Session

Next up: Memory and Observability